

THE SILICON VALLEY WORKFLOW

Week 1: The Hacker Setup

Post-Session Reference & Helping Material

Every Wednesday | 12:00 PM – 1:30 PM | Hassan Abidi Seminar Hall

1. Linux — Your Options

Linux is the operating system powering most of the world's servers, cloud infrastructure, and developer machines. Before learning commands, you need Linux on your machine. You have three options:

Method	What it is	Best for
WSL 2	Windows Subsystem for Linux — run Linux inside Windows without rebooting	Beginners on Windows. Easiest to set up.
Dual Boot	Install Linux alongside Windows on separate disk partition. Choose OS on boot.	Full native performance, serious Linux use.
VM	Virtual Machine via VirtualBox / VMware. Full Linux OS runs inside a window.	Testing distros, sandboxed environments.

2. Setting Up WSL 2

WSL 2 (Windows Subsystem for Linux 2) lets you run a real Linux kernel inside Windows. It integrates tightly with VS Code and works great for development.

Installation Steps

Open PowerShell as Administrator and run:

```
wsl --install
```

This installs WSL 2 with Ubuntu by default. After installation, restart your PC. On first launch, create a username and password.

Useful WSL Commands

```
wsl --list --verbose      # see installed distros and WSL version
wsl --set-default-version 2  # ensure WSL 2 is default
wsl --shutdown          # shutdown all running WSL instances
```

 **Tip:** You can open your WSL terminal directly from VS Code. Install the 'WSL' extension and click the green >< icon in the bottom-left corner.

3. Linux Navigation & Essential Commands

The terminal is your cockpit. Mastering navigation is the foundation of everything else.

Navigating the File System

Command	What it does
<code>pwd</code>	Print Working Directory — shows where you are
<code>ls</code>	List files in current directory
<code>ls -la</code>	List all files including hidden, with permissions
<code>cd folder_name</code>	Change into a directory
<code>cd ..</code>	Go up one directory level
<code>cd ~</code>	Go to your home directory
<code>cd /</code>	Go to root directory
<code>mkdir my_folder</code>	Create a new directory
<code>rmdir my_folder</code>	Remove an empty directory
<code>rm -rf folder/</code>	Force-remove a folder and everything in it (careful!)
<code>clear</code>	Clear the terminal screen

Working with Files

Command	What it does
<code>touch file.txt</code>	Create an empty file
<code>cat file.txt</code>	Display file contents in terminal
<code>nano file.txt</code>	Open file in nano text editor
<code>cp file.txt copy.txt</code>	Copy a file
<code>mv old.txt new.txt</code>	Rename or move a file
<code>rm file.txt</code>	Delete a file
<code>echo 'Hello' > file.txt</code>	Write text to a file (overwrites)
<code>echo 'Hello' >> file.txt</code>	Append text to a file
<code>grep 'text' file.txt</code>	Search for text inside a file

 **Tip:** Tab auto-completion is your best friend. Start typing a command or folder name and press Tab — Linux will complete it for you. Press Tab twice to see all options.

4. Git — Version Control Basics

Git is a distributed version control system. It tracks changes to your code so you can go back in time, collaborate, and never lose work.

Core Concepts

- Repository (repo)
- Commit
- Staging Area
- Branch

Initial Git Configuration (do this once)

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

The Git Workflow

Command	What it does
<code>git init</code>	Initialize a new Git repo in current folder
<code>git status</code>	See what files have changed / are staged
<code>git add file.txt</code>	Stage a specific file
<code>git add .</code>	Stage ALL changed files
<code>git commit -m "message"</code>	Save staged changes with a description
<code>git log</code>	View commit history
<code>git log --oneline</code>	Compact view of commit history
<code>git diff</code>	See unstaged changes line by line
<code>git restore file.txt</code>	Discard changes in a file (undo edits)

 **Tip:** Write commit messages that explain WHY a change was made, not just what. Good: 'Fix login bug when email has uppercase letters'. Bad: 'fixed stuff'.

5. GitHub — Remote Repositories

GitHub is a cloud platform that hosts Git repositories. It's where the world shares and collaborates on code. A local repo lives on your machine; a remote repo lives on GitHub.

Key GitHub Concepts

- Remote
- origin
- Push
- Pull
- Clone

Connecting Local Repo to GitHub (manual way)

1. Create a new repo on github.com (click + → New repository)
2. Copy the repo URL, then in your terminal:

```
git remote add origin https://github.com/yourusername/repo-name.git
git branch -M main
git push -u origin main
```

Common Remote Commands

Command	What it does
git remote -v	Show connected remotes
git push	Push commits to remote
git pull	Pull latest changes from remote
git clone <URL>	Clone a remote repo locally

6. GitHub CLI (gh) — Terminal to GitHub

The gh tool is GitHub's official command-line interface. It lets you authenticate your terminal with GitHub and perform GitHub actions (create repos, open PRs, etc.) without leaving the terminal.

Installing gh on Ubuntu / WSL

```
sudo apt update
sudo apt install gh
```

Authenticating with GitHub

```
gh auth login
```

Follow the prompts: select GitHub.com → HTTPS → Login with a web browser. A code will appear in the terminal — paste it in the browser window that opens. You are now authenticated!

Verify Authentication

```
gh auth status
```

Create a Repo and Push — The gh Way

Instead of manually creating a repo on GitHub's website, you can do it all from the terminal:

```
# 1. Initialize your project
git init
git add .
git commit -m "Initial commit"
# 2. Create GitHub repo and push in one command
gh repo create my-project --public --source=. --remote=origin --push
```

Other Useful gh Commands

Command	What it does
gh repo list	List all your GitHub repos
gh repo view	View current repo info in browser
gh repo clone username/repo	Clone any GitHub repo
gh issue list	List open issues in current repo
gh pr list	List open pull requests

7. The Complete Workflow — Start to Finish

Here is the complete workflow from zero to pushing code to GitHub, all in one place:

#	Command	Purpose
1	mkdir my-project && cd my-project	Create project folder and enter it
2	git init	Initialize Git tracking
3	touch README.md	Create a file
4	echo "# My Project" >> README.md	Add content to the file
5	git add .	Stage all changes
6	git commit -m "Initial commit"	Commit the snapshot
7	gh auth login	Authenticate terminal with GitHub
8	gh repo create my-project --public --source=. --remote=origin --push	Create GitHub repo and push
9	# Make more changes...	
10	git add . && git commit -m	Stage and commit new changes

	'Update'	
11	git push	Push to GitHub

8. Quick Reference Cheatsheet

Linux Navigation

```
pwd | ls | ls -la | cd folder | cd .. | cd ~ | mkdir | rm -rf
```

File Operations

```
touch file.txt | cat file | nano file | cp | mv | rm | echo 'text' > file
```

Git Essentials

```
git init | git status | git add . | git commit -m 'msg' | git log | git push | git pull
```

gh CLI

```
gh auth login | gh auth status | gh repo create | gh repo list | gh repo clone
```

WEEK 1 TASK

Build Your GitHub Profile README

Your GitHub profile is your developer identity. Make it count.

9. Your Task — GitHub Profile README

A GitHub Profile README is a special README.md that appears directly on your GitHub profile page — like a personal landing page for your developer identity. It is the first thing recruiters, collaborators, and the open-source community sees when they visit your profile.

What does it look like?

When someone visits `github.com/yourusername`, instead of an empty page they see your README rendered beautifully with your bio, skills, and projects. The reference image shows a real example of a polished profile README.

Step-by-Step Instructions

1

Create a repository named exactly after your username

Go to `github.com`, click the + button (top right), then New repository.

The repository name MUST be identical to your GitHub username.

Example: if your username is `hissamshar`, the repo must be named `hissamshar`

2

Set it to Public and initialize with a README

Check **Public** and tick **Add a README file**. GitHub will show a special message confirming this is a secret profile repo.

3

Clone it locally, edit README.md, then push using today's workflow

Use everything you learned today — no shortcuts through the GitHub web editor!

```
gh repo clone yourusername/yourusername
cd yourusername
nano README.md           # write your profile content here
git add .
git commit -m "Add profile README"
git push
```

Hint — Where does it show up?

Once pushed, visit github.com/yourusername — your README will appear at the top of your profile page, rendered as a webpage. No extra steps needed.

The magic path is: [yourusername](#) / [yourusername](#) / [README.md](#) — the repo and the username must match exactly.

What to Write in Your README

Section	What to include
About Me	Your name, university, year, and what you're studying or interested in
Currently Working On	Any projects, coursework, or self-learning you're doing right now
Currently Learning	New technologies you're picking up (e.g. AWS, Python, Git)
Skills	Languages and tools you know — you can use badge icons too
Reach Me	Email, LinkedIn, or any contact info you want to share
Fun Fact	Something personal — keeps your profile human and memorable

 **Tip:** Use github.com/rahuldkjain/github-profile-readme-generator — a free tool that builds your README for you. Fill in your details and paste the Markdown output into your README.md file.

Minimum Requirements

- A heading with your name
- A short About Me section
- At least one skill or technology you know
- Pushed via the git terminal workflow from today's session (not the GitHub web editor)

Coming Up Next — Week 2 (Feb 18)

Python & FastAPI

Learn Python for backend development, virtual environments, and building your first REST API server.

Got Questions? Stuck on Something?

Don't sit on a problem alone — that's what the community is for. Whether it's a command that isn't working, WSL refusing to install, a Git error you don't understand, or anything else from this session:

Post It in the Chat!

If you run into any issues, errors, or have questions after this session — post them in the group chat. No question is too small or too basic. We are all here to learn together.

When posting, try to include:

1. What you were trying to do
2. The exact command or step you ran
3. The error message you got (screenshot or copy-paste it)

The faster you share, the faster we can help. See you next Wednesday!

*AWS Cloud Club | FAST-National University Peshawar • Every Wednesday 12:00–1:30 PM • Hassan Abidi
Seminar Hall*